

Technical Note

Improved particle swarm optimization algorithm based novel encoding and decoding schemes for flexible job shop scheduling problem

Haojie Ding, Xingsheng Gu*

Key Laboratory of Advanced Control and Optimization for Chemical Processes, Ministry of Education, East China University of Science and Technology, Shanghai 200237, P. R. China

ARTICLE INFO

Article history:

Received 24 August 2019
 Revised 19 January 2020
 Accepted 31 March 2020
 Available online 3 May 2020

Keywords:

Flexible job shop scheduling problem
 Particle swarm optimization algorithm
 Encoding and decoding schemes
 Local search
 Operations research

ABSTRACT

The flexible job shop scheduling problem (FJSP) is a typical scheduling problem in practical production and has been proven to be a NP-hard problem. The study of FJSP is important to remarkably direct actual manufacturing processes. The paper proposes an improved particle swarm optimization (PSO) algorithm for solving FJSP and obtains beneficial solutions by improvement on encoding/decoding scheme, communication mechanism between particles, and alternate rules of candidate machines of operations. The innovation of encoding/decoding scheme proposes a novel designed chain encoding scheme and a corresponding effective decoding scheme. The chain-based encoding scheme can reasonably convert FJSP to an appropriate operation linked list and the novel designed decoding scheme owns the capacity of further exploring the solution space. The improvement of traditional PSO focuses on the innovation of information communication between particles, besides the modification of algorithm architecture. The amelioration of rules on operated machine selection is carried out based on the critical path of operations research (OR). It promotes algorithm efficiency by only alternating the candidate machines of operations on the critical path. In addition, much parameters tuning work is involved in a series of experiments. The study proposes some tuning schemes of parameters with exact mathematical methods, and these schemes can effectively help find more appropriate parameters. The final experiment results prove that the improved PSO exhibits remarkable ability to solve FJSP.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

The demand for personalized and customized commodities has been increasing, thereby causing rapid and unforeseen changes in processing customer orders in actual production, including decreasing volumes and increasing numbers of batches (Gothwal and Raj, 2018). To stay competitive, manufacturing companies have to rapidly respond to these unexpected changes in demand and produce commodities at reduced costs, which are accomplished through flexible and individualized production (Andrade-Pineda et al., 2019). Accordingly, flexible scheduling is taken to systematically manufacture commodities based on customer orders. Flexible scheduling enables the manufacturing system to be adaptive and valuable to actual production (Wang and Sun, 2009). The flexible job shop scheduling problem (FJSP) is drawn from the context of practical production to determine reasonable and feasible flexible scheduling plans; it has become a focused problem

in manufacturing research (Lu et al., 2017). Thus, solving FJSP is of great significance.

Production manufacturing fields involve numerous flexible processes, such as technological flexible process (Buzatu et al., 2009), and batch flexible process (Novara et al., 2016). This study focuses on the machine flexible process (also called path flexible process (Girish and Jawahar, 2009a)), which adds a process of arranging flexible machines into JSP (Gao et al., 2015). JSP is drawn from early production, which defines finishing one commodity produced on given facilities as a job and defines the facilities as machines. JSP describes a group of jobs that are assigned and processed on a set of given machines. A job also can be further divided into operations that are assigned to prescribed machines and are processed in order for being in accord with the specific technological setting (Shen et al., 2018). Classical JSP focuses on sequencing all the operations from all the jobs, given that every operation only has one candidate machine (Gao et al., 2018). However, several identical machines are often provided to perform similar operations to fulfill orders better in early production. These identical machines, which are called parallel machines, increase the flexibility and reliability of production processes and extend JSP to early FJSP. Furthermore,

* Corresponding author.

E-mail addresses: hero_ding1978@163.com (H. Ding), xsgu@ecust.edu.cn (X. Gu).

identical parallel machines are replaced with highly flexible machines owning different performance for different operations; thus, high flexibility becomes a characteristic of FJSP (Rossi, 2014). Correspondingly, all the machines that can process the same operation construct its candidate machine set. Given that highly flexible machines have different processing performance for different operations in FJSP, an operation often has varied processing times when performed on different machines of its candidate machine set. FJSP is more complex than JSP because of the addition of machine assignment. Given that JSP is strongly NP-hard (Garey et al., 1976), FJSP, as its extension, is also strongly NP-hard.

Different algorithms, including numerous meta-heuristic algorithms, have been used to solve FJSP in recent decades (Zhang et al., 2019). Meta-heuristics progressively explore the most promising regions of solution space for finding out high-quality solutions with well-designed heuristic rules; such algorithms are powerfully in solving scheduling problems (Pellerin et al., 2019). With the advantages of meta-heuristics and the growing computation power of modern computers, solving FJSP has considerably progressed (Deng and Gu, 2012).

Research on the heuristic algorithms for FJSP initially focused on the improvement of neighborhood structure and heuristic rules, such as tabu search (TS) (Brandimarte, 1993) and simulated annealing (SA) (Najid et al., 2002). Brandimarte proposed a TS-based hierarchical approach to solve FJSP (Brandimarte, 1993), in which a benchmark set was suggested and adopted. The benchmark set has been commonly used as a classical testing instance set. Hurink et al. constructed a disjunctive graph FJSP model for the minimum maximum completion time (makespan) criterion by introducing certain rules (Hurink et al., 1994). According to the critical path method from operations research (OR), critical operations on critical paths become the key elements of improving FJSP criterion. Although early neighborhood-based or rule-based heuristic algorithms have been successfully applied to solve FJSP, the insufficiency of diversities of solutions is increasingly obvious with the growth of FJSP scale. The study of FJSP addresses this insufficiency and gains enhanced achievements by ingeniously hybridizing one or more kinds of meta-heuristic algorithms and constraint rules from FJSP itself.

Many population-based meta-heuristic algorithms, such as genetic algorithm (GA) (Pezzella et al., 2008), ant colony optimization (ACO) (Chaoch et al., 2017), and particle swarm optimization (PSO) algorithm (Kennedy, 2010), have been developed since the 21st century. These algorithms produce diverse solutions to help solve FJSP by simulating the evolution process between generations. These solutions are useful for large-scale FJSP or JSP. Mesghouni et al. solved FJSP with GA (Mesghouni et al., 1997). Ong et al. adopted the clonal selection principle, which imitates the human immune system, to solve FJSP (Ong et al., 2005). Zribi et al. proposed a hierarchical method that combined GA and local search to solve FJSP (Zribi et al., 2007). Furthermore, Gao et al. hybridized a GA and bottleneck shifting (local search) procedure to solve a multi-objective FJSP (Gao et al., 2007). The bottleneck shifting was also derived from the critical path of OR. Tay and Ho designed a genetic programming (GP) approach to effectively evolve composite dispatching rules to help solve a multi-objective FJSP (Tay and Ho, 2008). Gu et al. designed a novel competitive co-evolutionary quantum genetic algorithm for stochastic job shop scheduling problem (Gu et al., 2010). In addition to GA, ACO has also been used to solve FJSP. Liouane et al. proposed an application of ACO combined with TS to solve FJSP for the minimum makespan criterion (Liouane et al., 2007). Girish and Jawahar compared a GA with an ACO in solving FJSP for the minimum makespan criterion. They found that ACO outperformed GA, and a constraint programming formulation was solved using the ILOG solver for various benchmark instances (Girish and Jawahar, 2009b).

In the past decade, attempts to solve FJSP with improved PSO have increased. Xia and Wu proposed a hierarchical approach for solving multi-objective FJSPs (Xia and Wu, 2005). The proposed approach solves FJSP by using the PSO algorithm to assign operations to machines and then applying the SA algorithm to schedule operations on each machine. The multi-objective is to minimize the makespan, the total workload of machines, and the workload of the most critical machine, simultaneously. Liu et al. proposed a hybrid meta-heuristic named variable neighborhood PSO, which combines variable neighborhood search (VNS) and PSO to solve a multi-objective FJSP (Liu et al., 2006). VNS is used to improve the local searching performance of PSO in every iteration. The criterion refers to the weighted sum of makespan time and the sum of flow time of all jobs. Jia et al. proposed a PSO algorithm to solve a multi-objective FJSP, and their method introduced chaos into the global search of PSO and made searches escape from local optima in a superior manner (Jia et al., 2007). The resulting precision and convergence rate are improved by embedding the chaotic local optimizer within PSO. The objective function simultaneously minimizes the makespan, the total workload of the machines, and the workload of the most loaded machine. Sha and Hsu proposed a PSO algorithm with preference list-based representation for solving JSP (Sha and Hsu, 2006); their designed preference list-based representation helped provide high quality results. Recently, Yuan and Xu proposed an effective multi-objective algorithm with a local search approach that is based on NSGA-II for a multi-objective FJSP (Yuan and Xu, 2013). Nouiri et al. investigated the distributed solving method of PSO for future implementation on embedded systems that can generate decisions in real time according to the state of resources and any unplanned or unforeseen event (Nouiri et al., 2018); their work mainly considered the practical application of PSO.

The literature review (Defraeye and Van Nieuwenhuyse, 2016) shows that hybridizing two or more kinds of meta-heuristic algorithms or criteria can achieve better results than only utilizing one kind of algorithm or criterion. Compared with other population-based meta-heuristic algorithms, PSO is simpler, more easily implemented, and more often applied to solve multi-objective FJSP through combination with other heuristic algorithms or criteria. However, existing research focuses more on the improvement and hybrids of heuristic algorithms based on several mature encoding schemes (Ho et al., 2007). These encoding schemes adopt either integrated or separated means of coding operation sequence information and machine assignment information, which are beneficial for solving FJSP in combination with certain meta-heuristic algorithms. Science breakthroughs commonly begin from the modification of basic components, and an encoding scheme is just a basic and essential part of heuristic algorithms.

Motivated to improve the basic encoding stage, this study proposes a novel encoding scheme. Meanwhile, an associated decoding scheme is designed based it. The encoding scheme adopts a method to integrate all the necessary information of scheduling one operation in a unit node and then links all the nodes to form a chain. This chain is released by a linked list technique and the scheduling information of each node is stored in a specially designed data structure. Every unit node of the chain is orderly established around each operation in a random operation sequence, which is formed by all the operations from all the jobs according to a procedure described in Section 3.3. The unit node is called the operation node, and the chain is called the operation list in this study. For the unit node directly associating with an operation, the unit node is simpler called the operation when causing no ambiguity in the paper. The novel encoding scheme can finally convert all the scheduling information of operations in any one case to an corresponding operation list, which can be decoded into a viable scheduling scheme (solution). For further converting an operation

list into a feasible scheduling solution, a novel decoding scheme is proposed in this study, which is elaborated in Section 3.2.2. Relying on the proposed decoding scheme, all the operations can be more reasonably dispatched with a sub-process of local search. The validity of the encoding and decoding schemes is further verified by embedding these schemes in our proposed improved PSO to solve FJSP.

The contributions of the study may be reflected in representing FJSP with the proposed encoding scheme, improving PSO algorithm with some proposed strategies, and further searching optimal solutions with the proposed decoding scheme.

- a) This study proposes a novel encoding scheme for representing FJSP. By encoding the necessary scheduling information of every operation into each node of an associated chain, an operation list is built. Further, the operation list is implemented by the pointer and linked list techniques of C++ program language and every operation list can be decoded into a viable scheduling scheme. Based the encoding scheme, adding, deleting, and adjusting some operation node orders are easily. Meanwhile, the proposed encoding scheme is suit for both dynamic and uncertain environments in actual productions.
- b) This study also proposes some beneficial strategies to help improve a traditional PSO. The improved PSO is a variation of the so-called rumor PSO algorithm (Clerc, 2010), which changes the communication mechanism of traditional PSO and has been well utilized in continuous problems. By adding an order element to the specially designed data structure of operation node, the FJSP, belongs to discrete problems, is successfully solved with the improved PSO. The improvement is mainly applied to solve the operation sequencing sublevel problem of FJSP. Meanwhile, the operated machine selection of every operation is connectedly thought over. After selecting out the critical operations with the critical path method from OR (Hu et al., 2018), these operated machines of the critical operations are tentatively reassigned for a better solution. During the innovational process of PSO, much work of tuning parameters is tested with mathematical methods, which makes these adopted parameters more precise and convinced.
- c) To convert an operation list into a feasible scheduling scheme, we propose a novel decoding scheme. The decoding scheme includes a local search process. With the local search process, it further explores the optimum of local solution space when dispatching a new operation to its assigned machine. Meanwhile, with the novel designed decoding scheme, any operation permutation of operation list can be accepted and be directly converted into a valid scheduling solution.

The remainder of the paper is organized as follows. The problem description and the formulation of FJSP are stated in Section 2. The proposed encoding scheme, decoding scheme, and the improvement of PSO for FJSP are separately elaborated in Section 3. The tuning work of related parameters is emerged in the anterior part of Section 4, and the corresponding experimental results are analyzed in the posterior part of Section 4. Finally, the conclusions and the future work are provided in Section 5.

2. Problem description and formulation

2.1. Problem description

FJSP is a typical combinatorial optimization problem drawn from actual industry production. Similar to JSP, FJSP defines producing one commodity as a job and considers the corresponding operated facilities as machines. Completing a job is usually divided into one independent process or several associated, manipulation-based independent, and consecutive processes. Every one of them

is assigned to a machine for execution and is defined as an operation. Essentially, operations correspond to the minimum indivisible elements or things to be scheduled (Giffler and Thompson, 1960).

FJSP falls into two categories based on the capacities of the candidate machine sets to process operations: total FJSP (T-FJSP) and partial FJSP (P-FJSP). As mentioned above the candidate machine set of one operation consists of all the machines, which can manipulate the operation. For T-FJSP, the candidate machine set of each operation includes all given machines; whereas for P-FJSP, the candidate machine set of each operation is equal to the entire set or a proper subset of all the available machines. In general, P-FJSP is an extension of T-FJSP and has a more universal research significance (Liu et al., 2007). Thus, P-FJSP is investigated in this paper and can be described as follows.

It is assumed that n independent jobs $J = \{J_1, J_2, \dots, J_n\}$ and m machines $M = \{M_1, M_2, \dots, M_m\}$ exist. Each job has an itself set of operations subjecting to precedence constraint relationships. All the jobs construct the total set of operations $O = \{O_{11}, O_{12}, \dots, O_{21}, O_{22}, \dots, O_{n1}, O_{n2}, \dots\}$. The j^{th} operation O_{ij} of job J_i may be assigned to any one machine M_k of its candidate machine set M_{ij} .

Given that various machines perform differently in processing varied operations, the processing time p_{ijk} of the operation O_{ij} is varied with the change of the operated machine M_k . An operation should be loaded on the assigned machine M_k at the start time S_{ij} , and be unloaded from the machine M_k at the end time E_{ij} . The processing time p_{ijk} is equal to the end time E_{ij} of the operation O_{ij} minus its start time S_{ij} on the assigned machine M_k . If the k^{th} machine M_k of the given entire machine set M can not process the operation O_{ij} , the start time S_{ij} and the end time E_{ij} of the operation O_{ij} on the k^{th} machine do not exist; accordingly, the processing time p_{ijk} also does not exist. In this study, we set the minimum maximum completion time (makespan) as the objective criterion to solve FJSP for its representative.

2.2. Problem formulation

This study formulates FJSP formulation by a set of notations. The parameter notations and the problem formulations are described briefly below.

(1) Parameters

n : total number of jobs

J : the complete set of all the jobs

i : index of jobs, $i = 1, 2, \dots, n$

J_i : the i^{th} job of J

O_i : the set of all the operations from J_i

n_i : total element number of O_i

j : index of operations of J_i , $j = 1, 2, \dots, n_i$

O_{ij} : the j^{th} operation of O_i

$O_{i(j+1)}$: the operation of following O_{ij} of J_i , where $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, (n_i - 1)$

m : total number of machines

M : the complete set of all the available machines

k : index of machines, $k = 1, 2, \dots, m$

M_k : the k^{th} machine of M

M_{ij} : the candidate machines set of operation O_{ij}

E_{ij} : end time of processing operation O_{ij}

S_{ij} : start time of processing operation O_{ij}

p_{ijk} : the processing time of operation O_{ij} on machine M_k

e : index of jobs, where $e \neq i$ and $e = 1, 2, \dots, n$

f : index of operations of J_e , $f = 1, 2, \dots, n_e$

x_{ijk} : decision variable representing whether operated machine M_k is assigned to operation O_{ij} ,

$$x_{ijk} = \begin{cases} 1, & \text{if machine } M_k \text{ is assigned to operation } O_{ij} \\ 0, & \text{otherwise} \end{cases}$$

y_{ijef} : decision variable representing the order of two different operations processed on the same machine,

$$y_{ijef} = \begin{cases} 1, & \text{if the operation } O_{ij} \text{ is processed before} \\ & \text{the operation } O_{ef} \text{ on the same machine} \\ 0, & \text{otherwise} \end{cases}$$

$C_{\max}$: the maximum completion time (i.e., makespan)

s : FJSP scale (i.e., sum of all operations), $s = \sum_{i=1}^n n_i$

F : freedom of FJSP, $F = \sum_{i=1}^n n_i / \sum_{i=1}^n \sum_{j=1}^{n_i} \text{Card}(M_{ij})$

where $\text{Card}(M_{ij})$ is the element number of set M_{ij}

(2) FJSP formulation

Criterion:

$$\min C_{\max} = \max\{E_{11}, \dots, E_{1n_1}; E_{21}, \dots, E_{2n_2}; \dots; E_{n1}, \dots, E_{nn_n}\} \quad (2.1)$$

Subject to:

$$S_{ij} + p_{ijk} \leq E_{ij} \quad (2.2)$$

$$E_{ij} \leq S_{i(j+1)} \text{ where } j+1 \leq n_j \quad (2.3)$$

$$E_{in_i} \leq C_{\max} \quad (2.4)$$

$$(S_{ij} + p_{ijk}) * y_{ijef} \leq S_{ef} \quad (2.5)$$

$$\sum_{k=1}^m x_{ijk} = 1 \quad (2.6)$$

In above FJSP formulation, the following assumptions are given. All the parameters take positive numbers or zero. Jobs are independent of each other, and no priorities are assigned to any job type. Job preemption or cancellation is not allowed, and all jobs are simultaneously available at time zero. Each job consists of one operation or several predefined operations, which are processed in predetermined order according to technological demands. However, there is not any precedence constraint between the operations from different jobs. Once an operation of any job is finished on its assigned machine, its attached job is transported to the next machine immediately; the transportation time is negligible. Meanwhile, breakdowns are not considered in this study. Eq. (2.1) represents the objective criterion subjected to the constraints of Eqs. (2.2)–(2.6) in this research. According to constraint (2.2), one execution must not be interrupted during the process of performing an operation on the assigned machine, once started. Constraint (2.3) confines that every operation is processed only after its predecessor is finished in the same job, that is, no more than one operation of every job can be operated at the same time. Constraint (2.4) prohibits the end time of any operation from being more than the maximum completion time $C_{\max}$. Constraint (2.5) confines that when two close operations are executed on the same machine, the successor can be processed only after the predecessor is finished. According to constraint (2.6), each operation can not be operated on more than one machine during the whole scheduling process.

3. Proposed improved PSO

The proposed improved PSO is designed based on the framework of the so-called rumor PSO algorithm with novel encoding and decoding schemes. The proposed PSO is further improved by hybridizing some methods to alternate the operated machines of the critical operations. These critical operations can be determined relying on the critical path method of OR with the aid of the FJSP constraints. The proposed improved PSO is described in Section 3.3.

3.1. Rumor PSO algorithm

This study proposes a variant of the tradition PSO algorithm to solve FJSP. The tradition PSO algorithm was inspired by

the cooperative behavior of birds in a swarm for seeking food (Kennedy, 2010). The tradition PSO algorithm considers every new position of one particle (bird) of swarm as a candidate solution and gives the particle some important attributes for describing the new position: the current position and velocity in the search space, the personal best position ever found out by the particle during its search process, and the current optimal global position found out by comparing all the personal best positions (Yao et al., 2019). Accordingly, the PSO algorithm also offers some turning parameters to approximate the true solution trajectories of problems. Through combination with previous iterative optimization methods and further improvements, two classical standard PSO equations (Bratton and Kennedy, 2007) are formed as follows.

$$v_{next} = w * v_{cur} + c_1 * r_1 * (p^* - x_{cur}) + c_2 * r_2 * (g^* - x_{cur}) \quad (3.1)$$

$$x_{next} = x_{cur} + v_{next} \quad (3.2)$$

where v_{cur} and x_{cur} respectively represent the velocity and position of current generation in iterations; v_{next} and x_{next} respectively represent the velocity and position of next generation; p^* represents the best personal position that one particle ever found out on its flying trajectory; and g^* represents the optimal position of swarm, which is obtained by comparing all the best personal positions from all the particles so far. w is an inertia weight, which is used to maintain the particle flight; c_1 is a personal confidence coefficient; and c_2 is a social confidence coefficient. r_1 and r_2 are independent random numbers generated between [0,1].

The rumor PSO, proposed by Clerc (Clerc, 2010), is a variant of tradition PSO, and has obtained some remarkable achievements in continuous problems. Differing from tradition PSO, each particle of rumor PSO takes remarkable information by comparing the position information of several neighborhood particles, but not directly takes the global best position information. The number of neighborhood particles is predefined as K , and $K = 3$ is adopted in this study according to Clerc's research results (Clerc, 2010). K particles are randomly selected from the entire swarm, and the objective information is communicated and spreaded from the K particles to the entire swarm with the continuously executing of iterations. The process of communicating information is similar to spreading a rumor, that is why the variant of PSO is named rumor PSO. Based on the manner, each particle can detect slight prospects of moves by obtaining the best position information of several neighborhood particles. Moreover, this information communication mechanism makes the motive trajectories of particles increasingly circuitous, which helps increase the diversity of solutions and the chance on the potential optimal solutions (Yao et al., 2019).

An order element is introduced to the data structure of operation node of operation list for applying the rumor PSO to solve FJSP, which is a typical NP-hard combinatorial optimum problem and is different from the continuous problems. An order element represents an inner position that an operation node is located an operation list, i.e., all operation nodes are always sorted in ascending order of the values of all the order elements. These values of order elements are iteratively computed with improved PSO Eqs. (3.3) and (3.4).

$$v_{next} = w * v_{cur} + c_1 * r_1 * (p^* - x_{cur}) + c_2 * r_2 * (g^K - x_{cur}) \quad (3.3)$$

$$x_{next} = x_{cur} + v_{next} \quad (3.4)$$

where, g^K is the best one of the K best personal positions. x_{cur} corresponds the current value of an order element, and x_{next} corresponds the new order value of the order element.

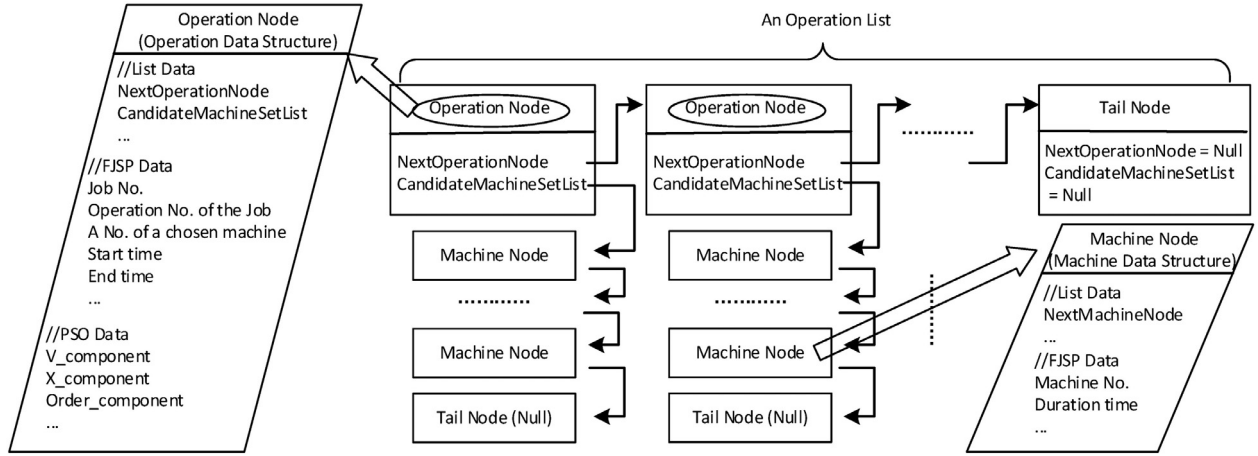


Fig. 1. Details of an operation list (particle).

3.2. Encoding and decoding schemes

3.2.1. Encoding scheme

FJSP includes two sublevel problems, namely, the operation sequencing sub-problem and the machine allocating sub-problem when dispatching an operation. Accordingly, an entire encoding scheme is designed around the two sublevel problems. The encoding scheme is in charge of converting all the operations into an operation list according to the formulations of FJSP, and all the related scheduling information also needs to be encoded in the operation list.

The improved PSO implements the novel encoding scheme relying on two well-designed data structures: one is called the operation data structure and another is named the machine data structure. Based the operation data structure, an linked list of operation list is built with the pointer and linked list techniques of the C++ program language. In the linked list of operation list, each pointer points an operation data structure associated with an operation and the pointed operation data structure includes some necessary elements, such as the handle address of the next operation data structure, the associated operation number, its attached job number, the handle address of the candidate machine set linked list, the chosen operated machine number, the related time parameters of scheduling the associated operation, the value of order element and other parameters for improved PSO equations. Meanwhile, all the possible machines that can operate the associated operation are selected out of all the given machines to establish the candidate machine set linked list. The candidate machine set linked list is also implemented with the pointer and linked list techniques based the machine data structure. In the linked list of candidate machine set, every machine data structure records some values of elements, such as the current candidate machine number, the processing time of the associated operation on the current candidate machine, and the handle address of the next machine structure. By the associated operation, a one-to-one relationship is established between every operation data structure of operation list and the corresponding candidate machine set linked list. Finally, the encoding work is finished and one possible schedule solution is represented by an operation list (Fig. 1), i.e., a position of particle is corresponded by an operation list filled with all the necessary values. The total node number of operation list is equal to the sum of all the operations, i.e., the scale s .

Table 1 represents a simple instance to expound the encoding scheme. The instance shows that two jobs are assigned to three machines to process. Job J_1 includes two operations noted O_{11} , and

Table 1

An instance of partial FJSP.

Job	Operation/ Processing time / Machine	M_1	M_2	M_3
J_1	O_{11}	4	-	4
	O_{12}	2	4	7
J_2	O_{21}	-	8	-
	O_{22}	-	-	5
	O_{23}	6	-	-
	O_{24}	7	-	6

O_{12} ; and job J_2 includes four operations noted O_{21} , O_{22} , O_{23} , and O_{24} . Every processing time of every operation on each machine is listed in Table 1 and - represents that the machine do not operate the prescribed operation. Fig. 2 illustrates a case of random assignments for the instance. O_{22} , O_{21} , O_{11} , O_{23} , O_{12} , O_{24} is a random operation permutation, and M_3 , M_2 , M_1 , M_1 , M_2 , M_3 is the corresponding operated machine permutation, which consists of one random chosen candidate machine of each operations candidate machine set. The case is that O_{22} , O_{21} , O_{11} , O_{23} , O_{12} , and O_{24} is respectively assigned to M_3 , M_2 , M_1 , M_1 , M_2 , and M_3 . Fig. 2 directly exhibit the case, and circle shadows of different gray levels are adopted to represent various operated machines. 5% of gray level, 25% of gray level, and 50% of gray level respectively represent M_1 , M_2 , and M_3 .

3.2.2. Decoding scheme

Once an operation list is generated with the proposed encoding scheme, a corresponding decoding scheme is required to convert the operation list into a feasible and appropriate scheduling scheme. In the study, a proposed novel designed decoding scheme is used to implement the transformation and generate a valid scheduling scheme.

The proposed novel designed decoding scheme authorizes a scheduled qualification to every operation according to the aforementioned assumptions and constraints of FJSP formulations. One operation that can be scheduled in current scheduling time is defined as a qualified operation and owns the authorized scheduled qualification, otherwise, the reverse. The definition shows that whether an operation is a qualified operation always changes with schedule generating, i.e. an operation is authorized different scheduled qualifications in different scheduling times. Since there is not any precedence constraint relation between two operations from different jobs and no more than one operation of every job can be operated at a given time, the maximum number of operations

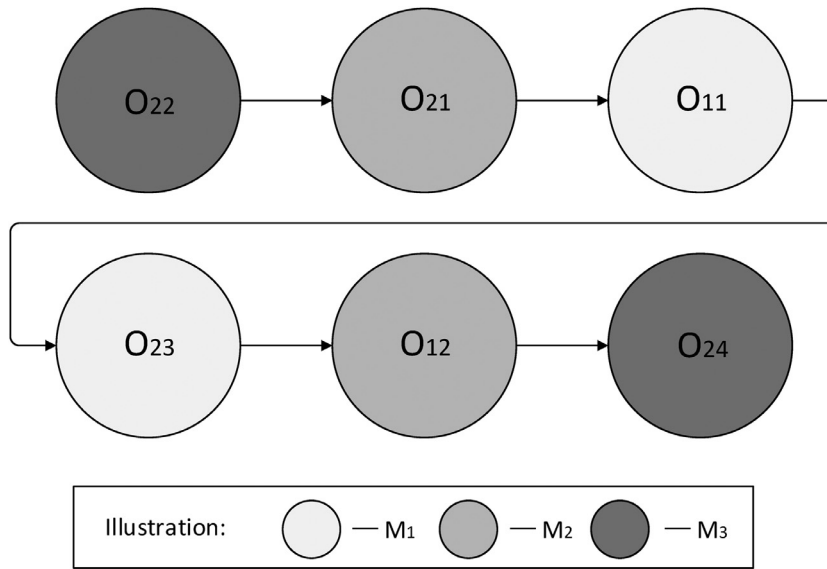


Fig. 2. A diagram for a random case of the instance listed in Table 1.

that can be scheduled is no more than the total number of jobs (n) at any scheduling time. Accordingly, the qualified operation number is also no more than n . During the decoding process, a local evaluation on the makespan of the next part-generated scheduling solution is executed for all the qualified operations when choosing and loading a new qualified operation on the allocated machine. The evaluation process is equal to a local search that finding out an appropriate operation from all the qualified operations to optimize the next part-generated scheduling solution; and the process of finding out the appropriate operation is implemented by tentatively loading each one of all the qualified operations on its assigned machine and comparing the tentatively obtained local makespans. All the qualified operations constructs a qualified operation set, which also changes with scheduling time. In the proposed novel decoding process, one local searching needs be executed before every adding a qualified operation to the current part-generated scheduling solution until all the operations are dispatched.

The procedure of the proposed novel decoding scheme is explicitly explained as follows.

Step 1 Establish and initialize a qualified operation set.

Step 1.1 Label the head operation of every job by traversing every operation node of operation list.

Step 1.2 Establish an empty set as an initial qualified operation set.

Step 2 Update the qualified operation set and execute an evaluation before every loading a qualified operation on its assigned machine until all the operations are dispatched.

Step 2.1 Place all the labeled operation in the qualified operation set. Then, check whether the qualified operation set is empty. If it is empty, then proceed to Step 3.

Step 2.2 Tentatively load every qualified operation on its assigned machine from the qualified operation set, and compute every tentative completion time. Then, record the operation O_{ij} of the minimum completion time.

Step 2.3 Load the chosen qualified operation O_{ij} on its assigned machine.

Step 2.4 Determine whether there is the succeeding operation of the chosen and just loaded operation O_{ij} in its attached job J_i .

Step 2.4.1 Determine whether j is less than n_i ; if they are equal, then proceed to Step 2.4.3.

Step 2.4.2 If j is less than n_i , then a succeeding operation exists. Identify and label the operation $O_{i(j+1)}$ by traversing every operation node of the remainder operation list. Meanwhile, cancel the label of operation O_{ij} . Reset the qualified operation set to an empty set, and then, proceed to Step 2.1.

Step 2.4.3 If j is equal to n_i , the operation O_{ij} is the tail operation of job J_i . Only cancel the label of the selected qualified operation O_{ij} . Then, reset the qualified operation set to an empty set, and implement Step 2.1.

Step 3 Stop the procedure, and return the entire schedule solution to the higher level procedure.

In the initialization of the proposed novel decoding scheme, an empty qualified operation set is built, and is updated with the head operation of each job. These operations is authorized the scheduled qualification by labelling them. With the scheduling solution growing, after an operation is loaded on its assigned machine, if a following operation exists in its attached job, the following operation is authorized the scheduled qualification by labelling it and then is placed in the updated qualified operation set. Meanwhile, the just loaded operation is cancelled the authorized qualify by cancelling the label. In the process of loading an appropriate operation on its assigned machine, the chosen operation always keeps the current completion time of the part-generated scheduling solution minimum. In particular, the qualified operation set is reset to an empty set after every dispatching an operation and is updated by placing all the new labeled qualified operations in it.

Given that each one operation of operation list can be dispatched to its operated machine only after that the operation has the scheduled qualification, the proposed novel decoding scheme convert an operation list into a feasible scheduling solution just in dispatched order. In fact, the dispatched order of operation is different from its position order in the operation list. Accordingly, the proposed novel decoding scheme can convert any operation permutation of operation list into a feasible scheduling solution. Thus, the randomness is completely retained, and the solution space is not compressed, too. Randomness is a basic and valuable feature to the population-based meta-heuristic algorithms and completely keeping it may lead the search to a promising cluster of solutions.

3.3. Proposed improved PSO algorithm

3.3.1. Improved PSO algorithm architecture

The size of particle swarm is assumed as N , and the entire framework of the proposed improved PSO algorithm is briefly introduced as follows.

Step 1 Initialization procedure

- Step 1.1 Generate an original operation list according to the data of the given instance file with the proposed encoding scheme, and respectively assign every operation to the first machine of its candidate machine set.
- Step 1.2 Set the count of the loops to 0. Then, generate N different particles to initialize the particle swarm.
 - Step 1.2.1 Determine whether the count of the loops reaches N . If it is not reached, add 1 to the count of the loops; if it is reached, proceed to Step 2.
 - Step 1.2.2 Copy the original operation list, and utilize the duplication as a new particle.
 - Step 1.2.3 Generate s random values, and respectively give every value to the order element in the operation data structure of each operation node of the duplication of the original operation list. Then, sort all the operation nodes by the ascending values of the order elements to form a new operation list (an initialized particle).
 - Step 1.2.4 Randomly assign an operated machine to every operation of the new operation list from the corresponding candidate machine set for forming a final initialized particle owning an initial position of solution space. Then, implement Step 1.2.1.
- Step 1.3 Make a load balancing with a balance procedure.

Step 2 Iteration procedure

- Step 2.1 Determine whether the upper limit of the preset iteration times is reached. If it is not reached, then set the count of the loops to 0; if it is reached, then proceed to Step 3.
- Step 2.2 Determine whether the count of the loops reaches N . If it is not reached, add 1 to the count of the loops; if it is reached, proceed to Step 2.1.
- Step 2.3 Randomly select K particles from the swarm and use the best fitness value of them as the g^k .
- Step 2.4 Update the value of order element of each operation node with the updated position variable generated with Eqs. (3.3) and (3.4). Sort all the operation nodes by the ascending values of their order elements to form a new operation list (a new position).
- Step 2.5 Evaluate the fitness value of the new operation list with the proposed novel decoding scheme; if the fitness value is better, replace the current record of p^* with the new operation list.
- Step 2.6 Alternate the operated machine of every critical operation from its candidate machine set, and the new operation list is changed, again.
- Step 2.7 Evaluate the fitness value of the newer operation list with the proposed novel decoding scheme again; if the fitness value is better, then replace the current record of p^* with the newer operation list. Proceed to Step 2.2.
- Step 3 Choose and output the optimal solution by comparing all the best personal positions.

The architecture of the improved PSO is simple and it generally consists of three procedures. The first is the initialization procedure, which generates all the particles and initializes them. The second is the iteration procedure, which continuously generates new operation lists owning better fitness values. The last one

chooses and outputs the optimal scheduling scheme by comparing all the best personal positions found in iteration procedure.

3.3.2. Initialization procedure

In the beginning of the initialization stage, the scheduling data of the instance file is associated to an original operation list. During the process of building the original operation list, the data is respectively given the corresponding elements of all the data structures. Meanwhile, every order element value, every position value, and every velocity value are set to zero. In the ending of the initialization stage, a balance procedure is executed to make a load balancing. The balance procedure finds out the maximum load machine of every particle, then randomly reassigns the operated machines of the operations on the maximum load machine. The balance procedure can help ensure rapider convergence during iteration procedure.

3.3.3. Iteration procedure

The iteration procedure adopts the improved PSO Eqs. (3.3) and (3.4) to help sequence all the operation nodes to form a new operation list. Fig. 3 explicitly illustrates the process of generating a new operation list with the instance exhibited in Fig. 2. The top half shows the current operation list with updated values and the bottom half emerges a next new operation list sorted by the updated values.

For every new generated operation list, the improved PSO uses the critical path method to identify the critical operations, then it adjusts the operated machine of every critical operation to form a newer operation list, which maybe have a better fitness value. The whole process is introduced as follows.

- Step 1 Copy the current operation list to generate its original duplication.
- Step 2 Convert the current operation list into a (backward) scheduling solution with the proposed novel decoding scheme.
- Step 3 Keep the current operation order of every machine of the just generated backward scheduling solution unchanged, rearrange every operation to its current operated machine as late as possible based the maximum completion time of the backward schedule, then generate a forward scheduling solution.
- Step 4 Determine the critical operations by comparing the start and end times of the same operation between the backward schedule and the forward schedule.
 - Step 4.1 Point the head node of the current operation list.
 - Step 4.2 Take out the pointed operation node and judge whether it is the tail node. If it is, then proceed to Step 5.
 - Step 4.3 If the two start times and the two end times of the pointed operation between the backward scheduling solution and the forward scheduling solution are separately equal, then label the operation as a critical operation.
 - Step 4.4 Point the next node of the current operation list, and then, implement Step 4.2.
- Step 5 Randomly alternate the operated machine of every critical operation $[m/2]$ times, meanwhile evaluate the fitness value after every change. If a better scheduling solution is obtained, update the best position p^* with the new obtained operation list and return it to the higher level procedure; otherwise, discard the current operation and return its original duplication to the higher level procedure.

where $[m/2]$ represents the minimal integer that is no less than the number $m/2$ in the pair of square brackets, and m is the total number of all the machines. Reviewing Step 5, when the times of alternating the candidate machines nears to 1, the variation of machine

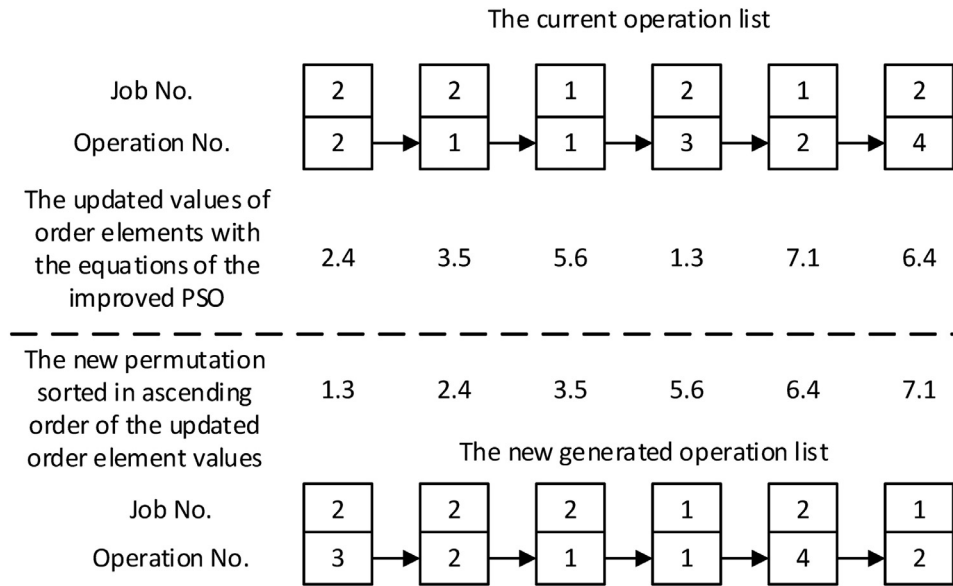


Fig. 3. The generating process of a new operation list.

assignment is too few to generate more various scheduling solutions, which maybe ignore some better solutions; when the times of alternating the candidate machines nears to m , the variation of machine assignment is too numerous to complete the searching in an expectant deadline, which makes the whole algorithm lose the applied values. Setting $\lfloor m/2 \rfloor$ as the times of alternating the candidate machines is a good compromise between the solution varieties and the computing duration.

The process of determining critical operations is illustrated in Fig. 4 by the case presented in Fig. 2. The top of Fig. 4 shows how to convert the current operation list into a (backward) scheduling solution, i.e., the decoding process. During the decoding process, the qualified operation set is updated with all the qualified operations after every reset to empty. Surveying the process, O_{21} and O_{11} have the qualifications to be scheduled in the beginning time 0; since O_{11} has the shorter completion time when be loaded on M_1 than O_{21} when be loaded on M_2 , the decoding scheme finally loads O_{11} on M_1 . After the qualified operation set reset to empty in next scheduling time (4), it is updated with O_{21} and O_{12} , and the similar procedures are iteratively executed, until the last qualified operation O_{24} is loaded in M_3 . Accordingly, a backward schedule is generated and the maximum completion time is 25. Then based on the backward scheduling solution and the maximum completion time (25), keep all the operated machines and orders of operations unchanged, all the operations are rearranged as late as possible to form the forward schedule, which is showed in bottom of Fig. 4. Comparing the backward schedule with the forward schedule, the operations surrounded by solid boxes are critical operations. Because their starting times and ending times are respectively equal in both the backward schedule and the forward schedule.

The fitness value of changed operation list is repeatedly evaluated throughout the whole iteration process. And the procedure of evaluating a fitness value is expounded as follows.

- Step 1 Convert a changed operation list into a scheduling solution by the proposed novel decoding scheme.
- Step 2 Identify the maximum end time by comparing all the end times of operations in the scheduling solution.

The maximum end time is the maximum completion time (makespan). Meanwhile, it is also the fitness value.

3.3.4. Choose and output the optimal scheduling solution

After all the iterations are finished, the optimal position of swarm is chosen and output by comparing all the best personal positions of particles. The optimal position of swarm corresponds to the operation list owning the best fitness value. With the proposed novel decoding scheme, the operation list owning the best fitness value can be converted into the optimal scheduling scheme. Furthermore, a Gantt chart can be plotted to directly survey the scheduling scheme like what Fig. 7 and Fig. 8 have done.

4. Experimental studies and discussion

The performance of the improved PSO algorithm is assessed in detail by studying the related parameters, convergence, and complexity and by comparing its results with those generated by other improved PSO algorithms. Our proposed improved PSO algorithm is coded with C++ program designed language. The experiments are implemented on a virtual machine with an Intel Core i7 @ 4.0 GHz, 16 GB RAM, and SUSE Linux Enterprise 12 operation system. The experimental procedures are edited with the Emacs editor and are compiled with a g++ compiler, which belong to the SUSE Linux OS. The convergence curves and the Gantt charts of the experimental results are plotted with MATLAB 16b.

4.1. Experiment arrangement and test instances

For surveying our proposed improved PSO, the experiment is carried out by three stages.

Firstly, we respectively test a small-scale problem, a medium-scale problem, and a large-scale problem with three instances taken from Motaghedi-larijani et al. (2010). The small-scale problem is a total FJSP with 4 jobs assigned to 5 machines (noted 4x5). The medium-scale problem is a partial FJSP with 8 jobs assigned to 8 machines (noted 8x8). The large-scale problem is a total FJSP with 15 jobs assigned to 10 machines (noted 15x10). Then, the study compares our results with some other results listed in the literature (Motaghedi-larijani et al., 2010). The three instances entirely test the performance of our proposed improved PSO when it is used to solve FJSP with all kinds of scales.

Secondly, for inspecting the effect of the proposed novel decoding schemes, we test Brandimartes benchmark set respectively using improved rumor PSO with a usual decoding scheme (noted

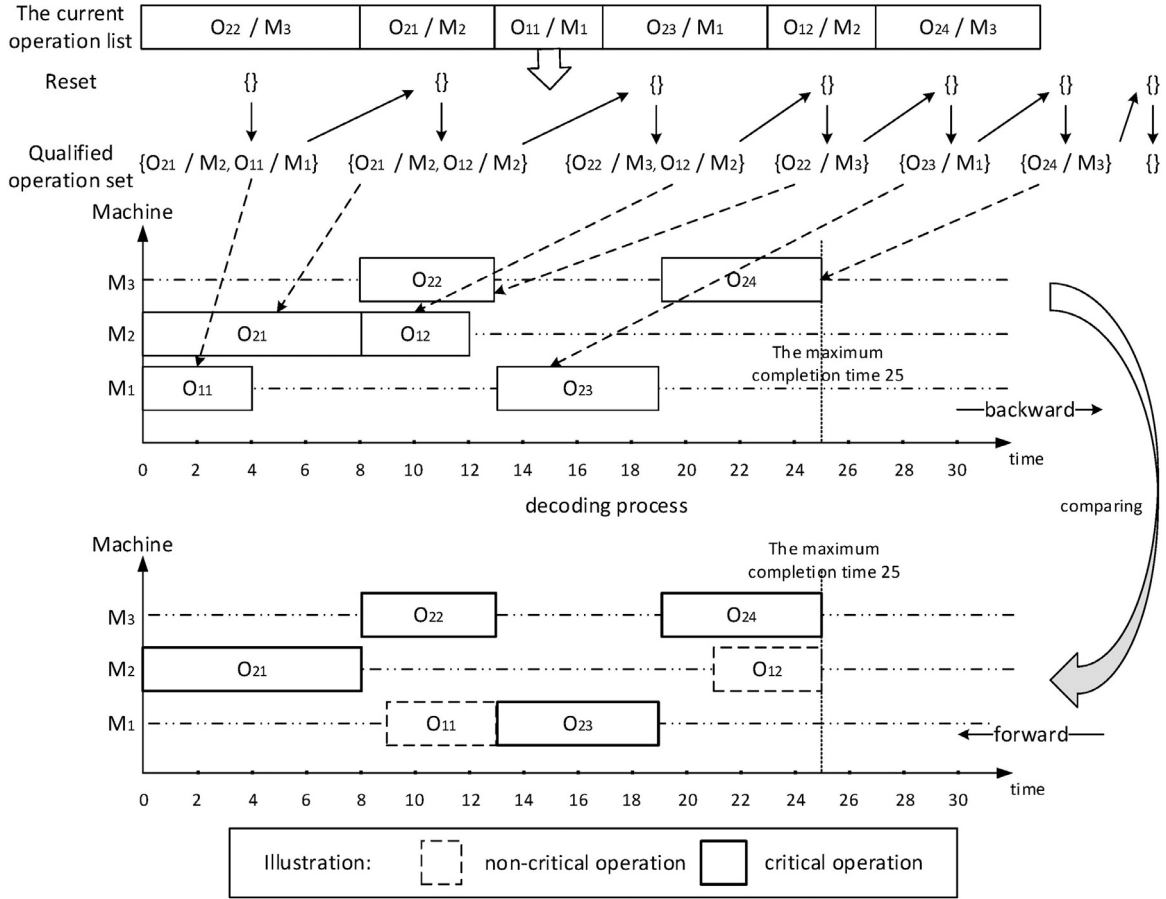


Fig. 4. A diagram for the processes of decoding and determining critical operations.

UDR PSO), a tradition PSO with our proposed novel decoding scheme (noted NDT PSO), and the proposed improved PSO (noted Proposed PSO). Then, all the results are put in together in comparison. Brandimartes benchmark set is proposed in his experiments (Brandimarte, 1993), which includes 10 instances that have different scales, freedoms, and technology backgrounds. Brandimartes benchmark set can comprehensively survey the performance of a heuristic algorithm and it have been adopted in most studies on FJSP as a universal and standard benchmark set.

Finally, we test Brandimartes benchmark set with the proposed improved PSO, and dividedly compare our results with the results obtained by other improved PSOs.

4.2. Parameters setting

The parameters can affect the capacity of algorithm. For the proposed improved PSO algorithm, only four parameters need to be tuned, and they have been denoted in Eqs. (3.3) and (3.4). According to the research results (Clerc, 2010): the inertia weight w is usually a random number generated between $[0,1]$; the personal confidence coefficient c_1 is generally equal to the social confidence coefficient c_2 , and they are proposed to take the classical value 1.42694 or 2.1; the number of informants K is proposed to set to 3 with considering both the good searching performance and the rapid convergence.

For further improving the searching capacity, the inertia weight w is proposed to make dynamic tuning with the increment of the iterations. Six kinds of dynamic tuning tendencies are designed to navigate w into the most promising value scope. Symbol $iteration$

denotes the current iteration, and $ITERATION$ represents the total iterations. These designed tendence trajectories include some convex curves, some concave curves, one line, and a random distribution between $[0,1]$. The tendence of parameter w on every kind of trajectory can be respectively expressed as follows with mathematic equations.

$$1\text{st trajectory (Exponent): } w = rand() * e^{\frac{iteration}{ITERATION} - 1},$$

$$2\text{nd trajectory (Parabola): } w = rand() * \left(\frac{iteration}{ITERATION}\right)^2,$$

$$3\text{rd trajectory (Line): } w = rand() * \left(\frac{iteration}{ITERATION}\right),$$

$$4\text{th trajectory (Circle): } w = rand() * \left[1 - \sqrt{1 - \left(\frac{iteration}{ITERATION}\right)^2}\right],$$

$$5\text{th trajectory (Square root): } w = rand() * \sqrt{\frac{iteration}{ITERATION}},$$

$$6\text{th trajectory (Random): } w = 0.689343 * rand().$$

Where $rand()$ is a random function provided in the library of the C++ program language for generating a random number between $[0,1]$, and the value 0.689343 is taken by the proposal from Clerc (2010).

The optimal experimental result and the mean of all the optimal results for every instance are listed in Table 2 and Table 3. They are obtained by independently running the improved PSO algorithm 5 times. The size of the swarm is tentatively set to 100, and the total number of iterations is set to 250.

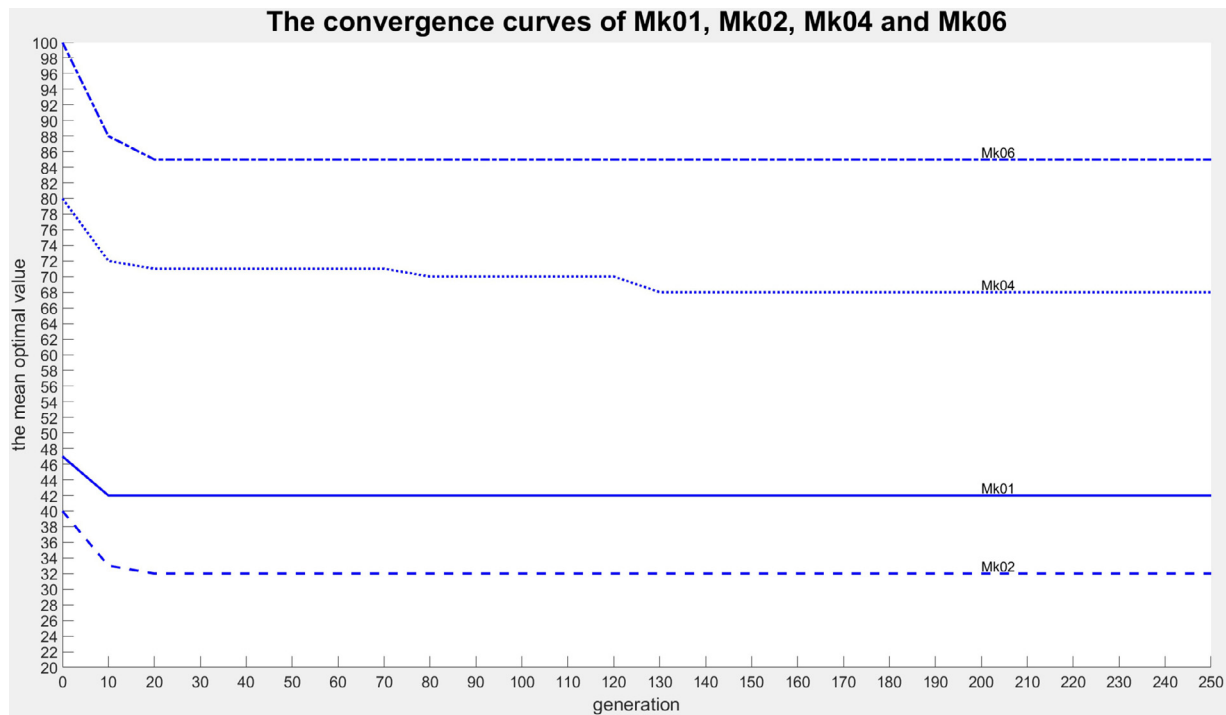
Surveying the results, the conclusion can be drawn that $c_1 = c_2 = 1.42694$ or 2.1, and w is generated with the 6th trajectory can lead to the better results. Accordingly, in the next experiments, parameter $w = 0.689343 * rand()$ is taken, and either of $c_1 = c_2 = 1.42694$ and $c_1 = c_2 = 2.1$ is randomly taken with an equiprobability.

Table 2Optimal and mean solutions of instances of *Mk01*, *Mk02*, *Mk04*, and *Mk05*.

trajectory of w ($w \in [0, 1]$)	<i>Mk01</i> ($c_1 = c_2$)				<i>Mk02</i> ($c_1 = c_2$)				<i>Mk04</i> ($c_1 = c_2$)				<i>Mk05</i> ($c_1 = c_2$)			
	1.42694		2.1		1.42694		2.1		1.42694		2.1		1.42694		2.1	
	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$
exponent	40	41.4	40	41.2	30	30.8	29	30.6	67	67.8	68	70	180	181.4	178	180
parabola	41	41.8	42	42	29	30.8	30	31.2	67	67.8	67	68.8	180	180.2	178	180
line	41	41.6	41	41.6	31	31.6	30	31.2	69	69.2	67	69	177	180.4	180	181
circle	42	42	41	41.8	30	31.2	30	31.2	67	68.4	68	69.6	178	179.8	180	181
square root	41	41.6	41	41.6	30	31.2	30	31.2	66	67.8	67	68.8	177	179.4	180	180.8
random	40	41.4	41	41.6	30	31.4	29	31	66	68.2	67	68.6	177	179.8	178	180.4

all the results of *Mk03* are 204.**Table 3**Optimal and mean solutions of instances of *Mk06*, *Mk07*, *Mk09*, and *Mk10*.

trajectory of w ($w \in [0, 1]$)	<i>Mk06</i> ($c_1 = c_2$)				<i>Mk07</i> ($c_1 = c_2$)				<i>Mk09</i> ($c_1 = c_2$)				<i>Mk10</i> ($c_1 = c_2$)			
	1.42694		2.1		1.42694		2.1		1.42694		2.1		1.42694		2.1	
	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$	Min	$\bar{M}$
exponent	80	84.6	82	83.8	147	148.2	148	148.6	334	343	331	340	260	267.4	356	261.2
parabola	84	85.2	83	85	149	150	148	149.2	336	340	335	341.2	250	259.8	259	265.4
line	85	86.2	79	83.8	149	149.8	147	148	335	344.6	337	346	261	265.4	255	259.2
circle	82	84.2	82	84.4	149	150	148	149	338	338.4	336	343.4	259	264	259	265.2
square root	83	85.6	82	84	147	148	148	149.2	336	341.8	337	341.6	255	264	258	264.2
random	82	83.8	81	84.4	147	148	145	148.6	339	343.8	326	347	258	264.6	261	264.2

all the results of *Mk08* are 523.**Fig. 5.** Convergence curves of *Mk01*, *Mk02*, *Mk04*, and *Mk06*.

4.3. Convergence analysis

The resulting curves of Brandimartes benchmarks are illustrated in Figs. 5 and Figs. 6 to show the convergence of the proposed improved PSO. The parameters are consistent with the conclusion parameters in Section 4.2. The proposed improved PSO is independently run 10 times, at every generation, the sum of all the 10 optimal values is averaged to generate the corresponding mean optimal value. Every resulting curve is drawn by the mean optimal value data and the generation data. Surveying these curves, the most of them converge after approximate 20 generations, and the

minority converge after about 100 generations, which indicates the convergence of the improved PSO can meet the demands of FJSP.

4.4. Complexity analysis

The complexity of the proposed improved PSO has close relationship with the scale of FJSP, i.e., the total number of operations. By surveying the duration of every iteration (be listed in Table 4) during the running process of the proposed improved PSO, the complexity of the proposed improved PSO can be accepted.

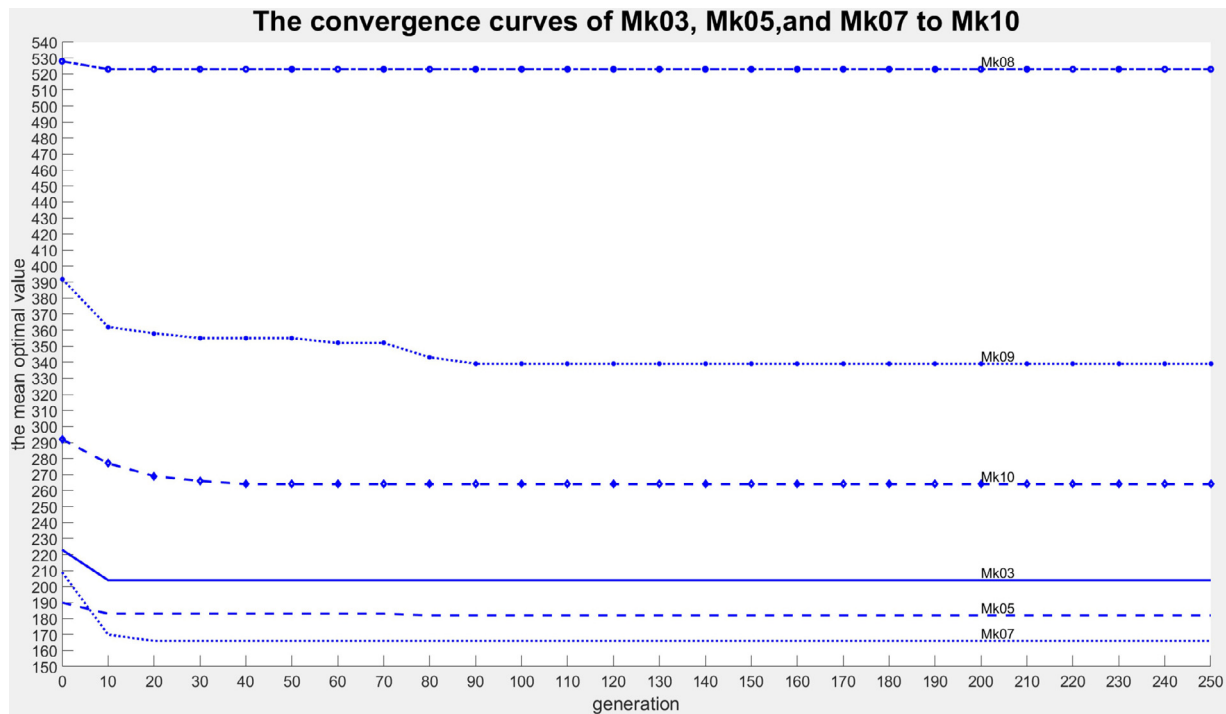


Fig. 6. Convergence curves of Mk01, Mk02, Mk04, and Mk06.

Table 4

Durations of every iteration of the proposed improved PSO.

Instance number	Total number of operations (s)	Duration of every iteration (unit: second)
Mk01	55	0.138838
Mk02	58	0.205846
Mk03	150	2.25404
Mk04	90	0.54175
Mk05	106	0.526314
Mk06	150	3.19145
Mk07	100	0.6425
Mk08	225	3.77957
Mk09	240	7.04822
Mk10	240	10.7187

Table 5

Comparison of results on Motaghedi-larijani's instances.

Instance (scale)	the optimal result interval based on Motaghedi-larijani's study	Proposed PSO (20 times)		
		min	max	mean
4x5	[11, 16]	11	11	11
8x8	[14, 19]	14	15	14.2
15x10	[11, 24]	13	15	14.1

4.5. Experiment result comparison and analysis

Before testing Brandimartes benchmarks, three instances from Motaghedi-larijani et al. (2010) are tested to survey the general performance of our proposed improved PSO. For clearly inspecting the performance of the algorithm, the total particle number N and iterations $ITERATION$ are reset for every instance and the other parameters are kept unchanged. For the small-scale (4x5) problem, $N = 5$ and $ITERATION = 10$ are taken; for the medium-scale (8x8) problem, $N = 20$ and $ITERATION = 50$ are taken; and for the large-scale (15x10) problem, $N = 100$ and $ITERATION = 500$ are taken. All the three instances are tested 10 times and the corresponding results are listed in Table 5 for comparing with the re-

sults from Motaghedi-larijani et al. (2010). Motaghedi-larijani et al. tested the three instances with many excellent algorithms in their experiments. Accordingly, the optimal result interval of every instance is listed in Table 5, and it includes all the optimal results generated by all kinds of algorithms mentioned in Motaghedi-larijani et al. (2010). From the comparison of results, the conclusion is drawn that the general performance of the proposed improved PSO approximates the outstanding heuristic algorithms used in Motaghedi-larijani et al. (2010).

Fig. 7 shows a Gantt chart for the Motaghedi-larijanis instance of 8x8 problem, and the optimal solution is obtained by using the proposed improved PSO. In the figure, p denotes the operated process of operation; the first number of the index pair in the following brackets represents a job number, and the second number represents the operation number of the attached job; and the value behind the equality sign corresponds to the processing time of operation on its allocated machine.

For comprehensively assessing the effect of the novel designed decoding schemes and the improved communication mechanism between particles, three different improved PSOs are experimented and compared each other. As mentioned in Section 4.1, UDR PSO points to the improved PSO based on the rumor communication mechanism with a usual active decoding scheme (Lu et al., 2017); NDT PSO points to the improved PSO based on the tradition

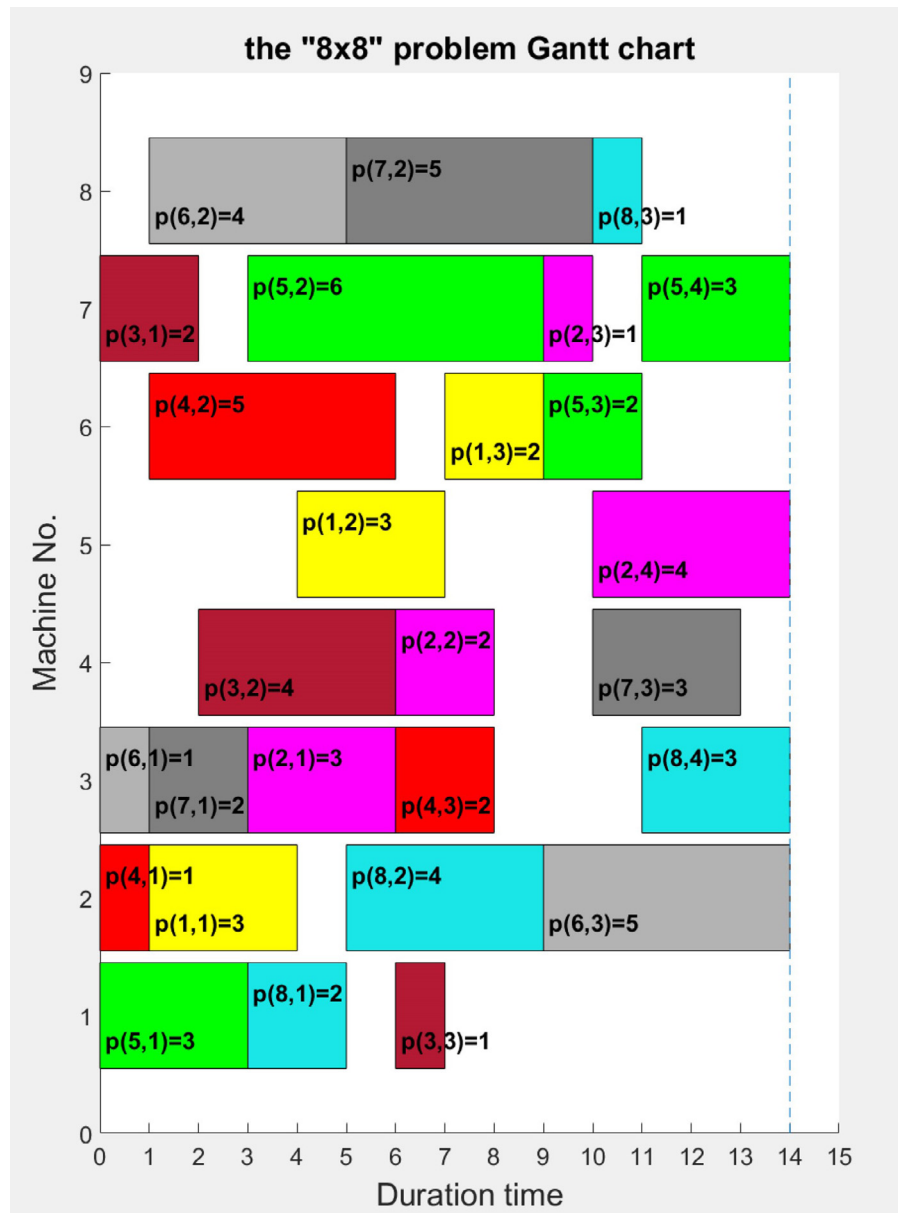


Fig. 7. the 8x8 problem Gantt chart of Motaghedi-Iarjani instances.

communication mechanism with the proposed novel designed decoding scheme; and Proposed PSO points the improved PSO based on the rumor communication mechanism with the proposed novel designed decoding scheme. These result data are listed in Table 6 for analysis and comparison. Meanwhile, the further result comparison with other improved PSOs is listed in Table 7. PSO + rules refers to the results from Girish and Jawahar (2009a); PSO + TS refers to the results from Li et al. (2010); and MATSPSO refers to the results from Henchiri and Ennigrou (2013). With the consideration on high quality solution and the convergence, N is set to 100 and $ITERATION$ is set to 500 in these experiments for Brandimartes benchmark set. While the parameters of tradition PSO in Eqs. (3.1) and (3.2) are taken $w = 0.729 * rand()$ and $c_1 = c_2 = 2$ by Ghosh et al. (2015). Because the tradition PSO does not involve a neighborhood searching, K has no relationship with the tradition PSO and needs not be set.

Surveying the comparison of results: UDR PSO obtains four optimal values by only improving the communication mechanism between particles, which adds a circuitous route to the traditional

searching trajectory; NDT PSO gets three optimal values by only using the proposed novel designed decoding scheme, which makes a further local search based on every new generated operation list; Proposed PSO achieves all the optimal values by adopting both of the above two improvements. Further, the results of *Mk01*, *Mk04*, *Mk05*, *Mk06*, *Mk09*, and *Mk10* show that both of the above two improvements bring beneficial effect in the process of searching the optimal solution. Because in the comparison of their results, only Proposed PSO obtain the optimal results. Continuously inspecting every maximum value column of Table 6, the every maximum result obtained by Proposed PSO is not more than the either one obtained by UDR PSO or NDT PSO. For every mean result column listed Table 6, it is similar. The exception that the result of *Mk02* indicates that metaheuristics based population belong to a probabilistic method and a small probability event may occur. After independently testing *Mk02* instance once more, combining the other mean results listed in Table 6, the conclusion is drawn that the every mean result of Proposed PSO is less than either one of both UDR PSO and NDT PSO. These conclusions further expounds

Table 6

Comparison of results on Brandimartes benchmarks with our improved PSOs.

Instance number	Instance scale	Size of operations	Instance freedom	UDR PSO (20 times)			NDT PSO (20 times)			Proposed PSO (20 times)		
				min	max	mean	min	max	mean	min	max	mean
Mk01	10x6	55	2	41	42	41.9	41	42	41.65	40	42	41.6
Mk02	10x6	58	3.5	29	32	30.2	29	32	30.9	29	32	31.1
Mk03	15x8	150	3	204	204	204	204	204	204	204	204	204
Mk04	15x8	90	2	67	70	68.5	67	71	68.9	66	70	68.05
Mk05	15x4	106	1.5	181	183	181.9	177	182	180.25	175	181	178.05
Mk06	10x15	150	3	78	82	79.9	81	88	84.1	77	84	80.7
Mk07	20x5	100	3	145	152	149.1	159	167	162.75	145	151	148.7
Mk08	20x10	225	1.5	523	523	523	523	523	523	523	523	523
Mk09	20x10	240	3	340	352	347.2	330	344	340.1	320	347	338.05
Mk10	20x15	240	3	251	260	257.2	254	267	261.7	239	256	248.85

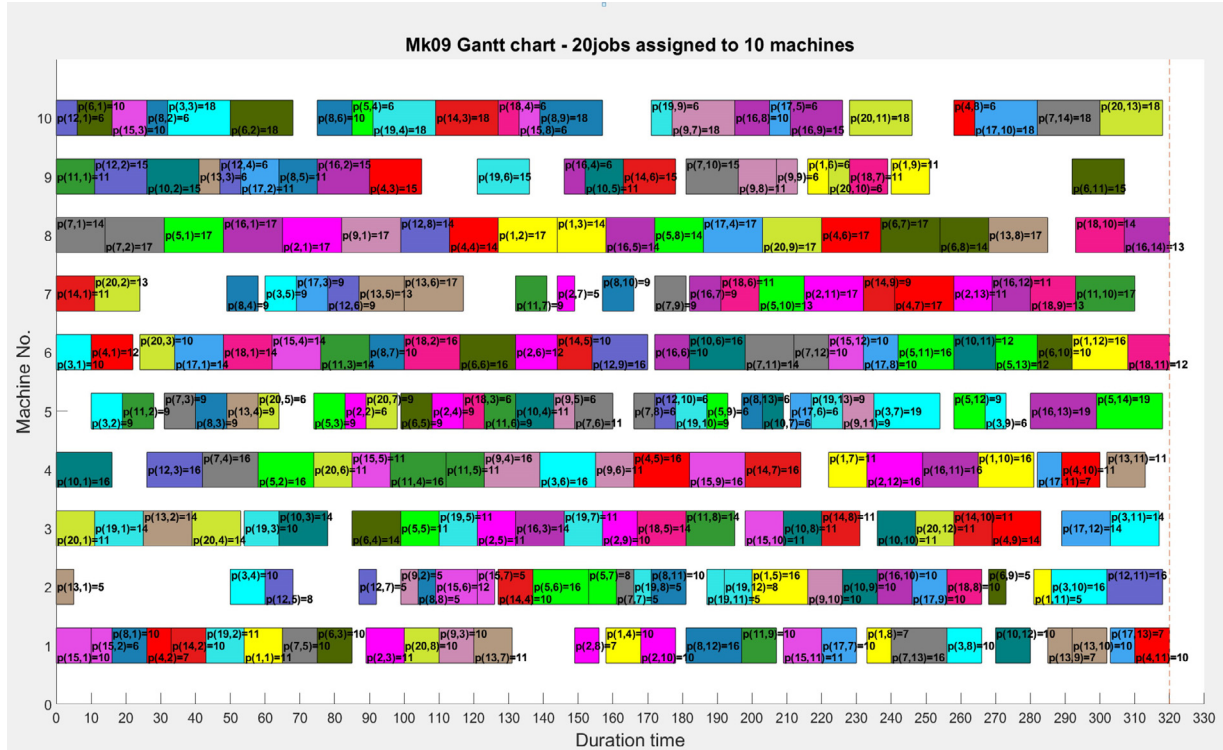
Best results compared with other methods are presented in bold and italic.

Table 7

Comparison of different PSO results on Brandimartes benchmarks.

Instance number	Instance scale	PSO +rules	PSO +TS	MATS-PSO	Proposed PSO
Mk01	10x6	40	40	39	40
Mk02	10x6	27	27	27	29
Mk03	15x8	204	204	207	204
Mk04	15x8	62	63	65	66
Mk05	15x4	178	173	174	175
Mk06	10x15	78	65	72	77
Mk07	20x5	147	145	154	145
Mk08	20x10	523	523	523	523
Mk09	20x10	341	331	340	320
Mk10	20x15	252	223	299	239

Our best results overcoming others are presented in bold and italic.

**Fig. 8.** Gantt chart of the optimal solution of Mk09 with the improved PSO algorithm.

that both of the two improvements promote the searching capacity of algorithm.

To evaluate the performance of the proposed improved PSO more objectively, three different designed improved PSOs are compared with the proposed improved PSO. The three PSOs are respectively named as PSO + rules, PSO + TS, and MATSPSO. PSO +

rules adopts a strategy of hybridizing tradition PSO with scheduling rules; PSO + TS makes a global search with traditional PSO and implements a further local search with TS; and MATSPSO searches the optimal solution by combining Multi-Agent technology with a hybrid of PSO and TS. The comparison of results on Brandimartes benchmarks is listed in Table 7. Surveying all the results, four

best of them can be obtained with our proposed improved PSO. Dividedly comparing our results with the results obtained by every other improved PSO, eight results of ours are better than or equal to the results from PSO + rules; five results of ours are better than or equal to the results from PSO + TS; and five results of ours are better than or equal to the results from MATSPSO. These comparison analysis further proves the efficiency of our proposed improved PSO.

Fig. 8 shows the *Mk09* instance Gantt chart, which corresponds the optimal scheduling solution obtained with our proposed improved PSO. The meaning of symbols is in accord with Fig. 7. The figure further illustrates the beneficial characteristics of our proposed improved PSO, namely, the small size of swarms, few iterations, and the finer searching capacity. These features help achieve an more appropriate and balance operation assignment.

5. Conclusion

As mentioned in the beginning, solving FJSP has an important significance. Based that, this study proposes an improved PSO for FJSP. Many novel ideas of algorithm improvement are applied to the whole process of algorithm running from the encoding/decoding stage to the searching optimal solution stage. After hybridizing them, the proposed improved PSO brings particles to a promising solution space, and implements a finer searching in a local solution area. According to the result data comparison and the experimental result analysis, the proposed improved PSO is proven effective and efficient. Meanwhile, it owns a simple architecture and low requirements to large-scale continuous memory space due to the chain encoding scheme, which is prone to be embedded in other scheduling softwares. Therefore, the proposed improved PSO algorithm is also suitable for more production and scheduling fields, such as the actual flexible manufacturing system (FMS).

Algorithm performance has a heavy relationship with the parameters. During the process of setting the related parameters, the reasonably tuning of parameters is proposed by means of mathematical methods, which has done a good preparation for further work. The current study focuses on a global optimization for the single objective FJSP. Besides single objective FJSP, multi-objective FJSP also numerously exists in actual industry production. That is another type of important problem. Much work of tuning parameters, making preference decision, and searching Pareto solutions needs be explored in multi-objective FJSP. Based on the current achievement of our proposed improved PSO, the further algorithm extension and improvement on multi-objective FJSP is also expected in the next work.

CRedit authorship contribution statement

Haojie Ding: Conceptualization, Methodology, Software, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Xingsheng Gu:** Methodology, Supervision, Project administration, Funding acquisition.

Acknowledgments

This study was financially sponsored by the [National Natural Science Foundation of China](#) (no. 61973120, no. 61573144, no. 61773165, and no. 61673175).

References

Andrade-Pineda, J.L., Canca, D., Gonzalez-R, P.L., Calle, M., 2019. Scheduling a dual-resource flexible job shop with makespan and due date-related criteria. *Ann. Oper. Res.* 1–31.
 Branimart, P., 1993. Routing and scheduling in a flexible job shop by tabu search. *Ann. Oper. Res.* 41 (3), 157–183.

Bratton, D., Kennedy, J., 2007. Defining a standard for particle swarm optimization. In: 2007 IEEE swarm intelligence symposium. IEEE, pp. 120–127.
 Buzatu, C., Fota, A., Lepadatescu, B., Duicu, S., 2009. Modelling by finite element of the part-tool flexible technological system deformations at superfinishing process. In: Proceedings of the 8th WSEAS international conference on Artificial intelligence, knowledge engineering and data bases. World Scientific and Engineering Academy and Society (WSEAS), pp. 139–142.
 Chaouch, I., Driss, O.B., Ghedira, K., 2017. A modified ant colony optimization algorithm for the distributed job shop scheduling problem. *Procedia Comput. Sci.* 112, 296–305.
 Clerc, M., 2010. Particle swarm optimization, Vol. 93. John Wiley & Sons.
 Defraeye, M., Van Nieuwenhuysse, I., 2016. Staffing and scheduling under nonstationary demand for service: a literature review. *Omega* 58, 4–25.
 Deng, G., Gu, X., 2012. A hybrid discrete differential evolution algorithm for the no-idle permutation flow shop scheduling problem with makespan criterion. *Comput. Oper. Res.* 39 (9), 2152–2160.
 Gao, J., Gen, M., Sun, L., Zhao, X., 2007. A hybrid of genetic algorithm and bottleneck shifting for multiobjective flexible job shop scheduling problems. *Comput. Ind. Eng.* 53 (1), 149–162.
 Gao, K., Yang, F., Zhou, M., Pan, Q., Suganthan, P.N., 2018. Flexible job-shop rescheduling for new job insertion by using discrete jaya algorithm. *IEEE Trans Cybern* 49 (5), 1944–1955.
 Gao, K.Z., Suganthan, P.N., Chua, T.J., Chong, C.S., Cai, T.X., Pan, Q.K., 2015. A two-stage artificial bee colony algorithm scheduling flexible job-shop scheduling problem with new job insertion. *Expert Syst. Appl.* 42 (21), 7652–7663.
 Garey, M.R., Johnson, D.S., Sethi, R., 1976. The complexity of flowshop and jobshop scheduling. *Math. Oper. Res.* 1 (2), 117–129.
 Ghosh, S., Nath, S., Sarkar, S.K., 2015. Pso algorithm with self tuned parameter for efficient routing in vlsi design. In: Proceedings of SSRG-IJEE International Conference on Futuristic Trends in Computing and Communication, pp. 60–63.
 Giffler, B., Thompson, G.L., 1960. Algorithms for solving production-scheduling problems. *Oper. Res.* 8 (4), 487–503.
 Girish, B.S., Jawahar, N., 2009. A particle swarm optimization algorithm for flexible job shop scheduling problem. In: 2009 IEEE International Conference on Automation Science and Engineering. IEEE, pp. 298–303.
 Girish, B.S., Jawahar, N., 2009. Scheduling job shop associated with multiple routings with genetic and ant colony heuristics. *Int. J. Prod. Res.* 47 (14), 3891–3917.
 Gothwal, S., Raj, T., 2018. Prioritising the performance measures of FMS using multi-criteria decision making approaches. *Int. J. Proc. Manag. Benchmark.* 8 (1), 59–78.
 Gu, J., Gu, M., Cao, C., Gu, X., 2010. A novel competitive co-evolutionary quantum genetic algorithm for stochastic job shop scheduling problem. *Comput. Oper. Res.* 37 (5), 927–937.
 Henschir, A., Ennigrou, M., 2013. Particle swarm optimization combined with tabu search in a multi-agent model for flexible job shop problem. In: International Conference in Swarm Intelligence. Springer, pp. 385–394.
 Ho, N.B., Tay, J.C., Lai, E.M.-K., 2007. An effective architecture for learning and evolving flexible job-shop schedules. *Eur. J. Oper. Res.* 179 (2), 316–333.
 Hu, J., Zhou, Y., Wei, Y., Quay, S., Reddy, L., Tellez, G., Nam, G.-J., 2018. Interconnect optimization considering multiple critical paths. In: Proceedings of the 2018 International Symposium on Physical Design. ACM, pp. 132–138.
 Hurink, J., Jurisch, B., Thole, M., 1994. Tabu search for the job-shop scheduling problem with multi-purpose machines. *Opera.-Res.-Spektrum* 15 (4), 205–215.
 Jia, Z., Chen, H., Tang, J., 2007. An improved particle swarm optimization for multi-objective flexible job-shop scheduling problem. In: 2007 IEEE International Conference on Grey Systems and Intelligent Services. IEEE, pp. 1587–1592.
 Kennedy, J., 2010. Particle swarm optimization. *Encycl. Mach. Learn.* 760–766.
 Li, J.-q., Pan, Q.-k., Xie, S.-x., Jia, B.-x., Wang, Y.-t., 2010. A hybrid particle swarm optimization and tabu search algorithm for flexible job-shop scheduling problem. *Int. J. Comput. Theory Eng.* 2 (2), 189.
 Liouane, N., Saad, I., Hammadi, S., Borne, P., 2007. Ant systems & local search optimization for flexible job shop scheduling production. *Int. J. Comput. Commun. Control* 2 (2), 174–184.
 Liu, H., Abraham, A., Choi, O., Moon, S.H., 2006. Variable neighborhood particle swarm optimization for multi-objective flexible job-shop scheduling problems. In: Asia-Pacific Conference on Simulated Evolution and Learning. Springer, pp. 197–204.
 Liu, H., Abraham, A., Grosan, C., 2007. A novel variable neighborhood particle swarm optimization for multi-objective flexible job-shop scheduling problems. In: 2007 2nd International Conference on Digital Information Management, Vol. 1. IEEE, pp. 138–145.
 Lu, C., Li, X., Gao, L., Liao, W., Yi, J., 2017. An effective multi-objective discrete virus optimization algorithm for flexible job-shop scheduling problem with controllable processing times. *Comput. Ind. Eng.* 104, 156–174.
 Mesghouni, K., Hammadi, S., Borne, P., 1997. Evolution programs for job-shop scheduling. In: 1997 IEEE International Conference on Systems, Man, and Cybernetics. Computational Cybernetics and Simulation, Vol. 1. IEEE, pp. 720–725.
 Motaghedi-larijani, A., Sabri-laghaie, K., Heydari, M., 2010. Solving flexible job shop scheduling with multi objective approach. *Int. J. Ind. Eng. Product. Res.*
 Najid, N.M., Dauzere-Peres, S., Zaidat, A., 2002. A modified simulated annealing method for flexible job shop scheduling problem. In: IEEE international conference on systems, man and cybernetics, Vol. 5. IEEE, pp. 6–pp.
 Nouiri, M., Bekrar, A., Jemai, A., Niar, S., Ammari, A.C., 2018. An effective and distributed particle swarm optimization algorithm for flexible job-shop scheduling problem. *J. Intell. Manuf.* 29 (3), 603–615.

- Novara, F.M., Novas, J.M., Henning, G.P., 2016. A novel constraint programming model for large-scale scheduling problems in multiproduct multistage batch plants: limited resources and campaign-based operation. *Comput. Chem. Eng.* 93, 101–117.
- Ong, Z.X., Tay, J.C., Kwok, C.K., 2005. Applying the clonal selection principle to find flexible job-shop schedules. In: *International Conference on Artificial Immune Systems*. Springer, pp. 442–455.
- Pellerin, R., Perrier, N., Berthaut, F., 2019. A survey of hybrid metaheuristics for the resource-constrained project scheduling problem. *Eur. J. Oper. Res.*
- Pezzella, F., Morganti, G., Ciaschetti, G., 2008. A genetic algorithm for the flexible job-shop scheduling problem. *Comput. Oper. Res.* 35 (10), 3202–3212.
- Rossi, A., 2014. Flexible job shop scheduling with sequence-dependent setup and transportation times by ant colony with reinforced pheromone relationships. *Int. J. Prod. Econ.* 153, 253–267.
- Sha, D.Y., Hsu, C.-Y., 2006. A hybrid particle swarm optimization for job shop scheduling problem. *Comput. Ind. Eng.* 51 (4), 791–808.
- Shen, L., Dauzère-Pérès, S., Neufeld, J.S., 2018. Solving the flexible job shop scheduling problem with sequence-dependent setup times. *Eur. J. Oper. Res.* 265 (2), 503–516.
- Tay, J.C., Ho, N.B., 2008. Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Comput. Ind. Eng.* 54 (3), 453–473.
- Wang, R., Sun, B., 2009. Design of flexible scheduling system based on an improved genetic algorithm. In: *2009 Second International Symposium on Computational Intelligence and Design*, 2. IEEE, pp. 312–315.
- Xia, W., Wu, Z., 2005. An effective hybrid optimization approach for multi-objective flexible job-shop scheduling problems. *Comput. Ind. Eng.* 48 (2), 409–425.
- Yao, L., Liu, Y., Zhao, H., Ding, H., 2019. An improved UKPK-PSO algorithm inspired from block chain technology for flexible job shop scheduling problem. In: *2019 Chinese Control Conference (CCC)*. IEEE, pp. 2260–2265.
- Yuan, Y., Xu, H., 2013. Multiobjective flexible job shop scheduling using memetic algorithms. *IEEE Trans. Autom. Sci. Eng.* 12 (1), 336–353.
- Zhang, J., Ding, G., Zou, Y., Qin, S., Fu, J., 2019. Review of job shop scheduling research and its new perspectives under industry 4.0. *J. Intell. Manuf.* 30 (4), 1809–1830.
- Zribi, N., Kacem, I., El Kamel, A., Borne, P., 2007. Assignment and scheduling in flexible job-shops by hierarchical optimization. *IEEE Trans. Syst. Man Cybernet. Part C (Appl. Rev.)* 37 (4), 652–661.